

# TP 2

## Administration distante et filtrage réseau en double pile

Modalité Travail en groupe de 2 ou 3 étudiants

### Contexte et finalité

Le laboratoire universitaire autorise l'administration d'un serveur depuis des sources identifiées. Vous établissez la confiance SSH, vérifiez la politique effective, filtrez IPv4 et IPv6 et observez une réponse automatique aux échecs. Le parcours privilégie des tests contrôlés avec un témoin positif et un témoin négatif.

### Objectifs pédagogiques

- Distinguer confiance dans l'hôte, authentification par clé et autorisation par groupe.
- Contrôler la configuration effective OpenSSH et vérifier des connexions nouvelles.
- Appliquer et éprouver un filtrage IPv4/IPv6 sans supprimer le trafic de contrôle nécessaire.
- Corréler un échec SSH, un bannissement et le retour à un accès légitime.
- Adapter puis révoquer un accès temporaire avec des preuves limitées et pertinentes.

### Prérequis et environnement technique

- Travail en groupe de 2 ou 3 ; alterner les rôles opérateur serveur, opérateur client et vérificateur. Un seul rapport PDF par groupe.
- Deux VM Ubuntu Server 24.04 LTS x86\_64 déjà installées et démarrées, avec accès sudo, console hyperviseur et Internet pour les paquets. Les commandes de base du shell sont acquises.
- Chaque VM possède une carte NAT sortante et une carte sur le même réseau interne TP2-SEC. Aucun pont vers le réseau réel ni redirection entrante NAT. Le serveur part sans autre pare-feu actif.
- Les adresses et règles du TP sont temporaires ; aucun redémarrage pendant la séance. Les consoles restent accessibles indépendamment de SSH. Seules ces VM sont ciblées.
- Les deux prolongements sont facultatifs et ne sont pas évalués.

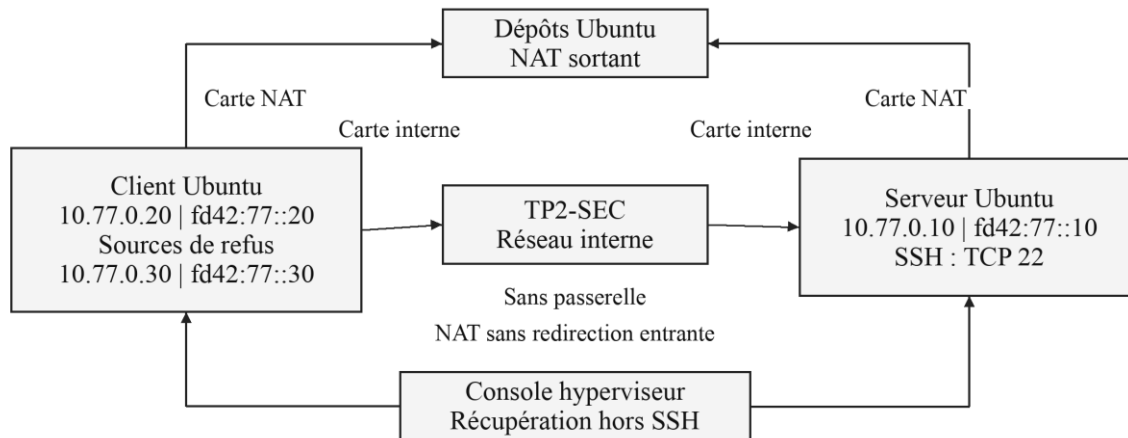
### Travail demandé

Réalisez les manipulations guidées, répondez aux questions numérotées et conservez les preuves demandées. Traitez ensuite la tâche autonome et présentez vos résultats dans un rapport PDF. Les prolongements facultatifs ne sont pas évalués.

# 1 Réseau expérimental isolé

## 1.1 Topologie et environnement

Utiliser deux VM déjà installées et démarrées : une carte NAT sortante et une carte interne TP2-SEC par VM, sans pont ni redirection entrante. Garder les consoles ouvertes et créer un instantané. Le serveur est une VM propre, sans règles nftables ni UFW actif. Les adresses internes seront temporaires : ne pas redémarrer pendant ce TP.



**Figure 1 Réseau de manipulation et adresses sources de test**

## 1.2 Outils et adressage

Sur les deux VM, installer les outils clients. L'installation du serveur SSH aura lieu au début de la section 2.

```
sudo apt update
sudo apt install openssh-client netcat-openbsd iputils-ping
ssh -V
```

Sur chaque VM, identifier la carte interne par son adresse MAC dans l'hyperviseur. Ne pas modifier la carte portant la route NAT par défaut. Saisir les valeurs de la figure : serveur .10 / ::10, client .20 / ::20. La carte interne doit être libre de configuration contradictoire.

```
ip -br addr
ip route show default
read -r -p "Carte interne : " LAB_IF
read -r -p "Adresse IPv4 interne : " LAB_V4
read -r -p "Adresse IPv6 interne : " LAB_V6
sudo ip link set "$LAB_IF" up
sudo ip address add "$LAB_V4/24" dev "$LAB_IF"
sudo ip -6 address add "$LAB_V6/64" dev "$LAB_IF"
```

## 1.3 Connectivité et analyse

Depuis le client, exécuter `ping -c 2 10.77.0.10` puis `ping -6 -c 2 fd42:77::10`. Attendre la fin de la détection de doublon IPv6 : `ip -6 addr` ne doit plus afficher tentative ni dadfailed pour les adresses du TP.

**Q1** À partir de la figure 1, expliquer le rôle du NAT sortant, l'absence de passerelle interne et le moyen de récupération si SSH devient inaccessible.

**P1 Preuve à conserver** Une topologie annotée avec les noms d'interfaces et un extrait des adresses et des deux ping.

## 2 Confiance SSH et authentification par clé

### 2.1 Service SSH et comptes

À la console du serveur, installer et démarrer SSH. Fail2ban reste arrêté jusqu'à la section 6. Vérifier que le serveur part sans règles nftables ni UFW actif.

```
sudo apt install openssh-server nftables fail2ban python3-systemd
sudo systemctl disable --now fail2ban ssh.socket
sudo systemctl enable --now ssh.service
sudo nft list ruleset
sudo ufw status
```

Sur le serveur, créer les comptes de test avec des mots de passe temporaires saisis interactivement. Seul tpadmin appartient au groupe autorisé ; aucun compte de test ne reçoit sudo. La clé du même client sera aussi installée pour tpexterne afin de tester ensuite le refus dû au groupe.

```
sudo addgroup sshadmin
sudo adduser tpadmin
sudo adduser tpexterne
sudo usermod -aG sshadmin tpadmin
sudo ssh-keygen -lf/etc/ssh/ssh_host_ed25519_key.pub -E sha256
```

### 2.2 Vérification de l'hôte et déploiement des clés

Sur le client, noter l'empreinte affichée à la console serveur. Générer une nouvelle clé avec phrase secrète dans un fichier encore inexistant. À chaque premier accès, comparer l'empreinte du serveur à cette valeur avant d'accepter ; arrêter si elle diffère [R1, R2].

```
install -d -m 700 ~/.ssh
ssh-keygen -t ed25519 -a 64 -f ~/.ssh/id_ed25519_tp2
ssh-copy-id -i ~/.ssh/id_ed25519_tp2.pub \
-o HostKeyAlgorithms=ssh-ed25519 tpadmin@10.77.0.10
ssh-copy-id -i ~/.ssh/id_ed25519_tp2.pub \
-o HostKeyAlgorithms=ssh-ed25519 tpexterne@10.77.0.10
```

Si le serveur interdit déjà les mots de passe, transférer uniquement la ligne .pub par la console dans le fichier ~/.ssh/authorized\_keys de chaque compte : propriétaire correspondant, répertoire .ssh en 700, fichier en 600. Ne jamais transférer la clé privée.

## 2.3 Validation des connexions par clé

```
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519_tp2
ssh -i ~/.ssh/id_ed25519_tp2 -o IdentitiesOnly=yes \
  tpadmin@10.77.0.10 id
ssh -6 -i ~/.ssh/id_ed25519_tp2 -o IdentitiesOnly=yes \
  -o HostKeyAlgorithms=ssh-ed25519 tpadmin@fd42:77::10 id
```

Comparer aussi l'empreinte au premier accès IPv6. Conserver la console serveur et une session SSH valide ouvertes. L'agent évite la répétition de la phrase secrète pendant les essais ; ne pas activer son transfert au serveur.

**Q2** Distinguer la clé d'hôte, la clé publique de l'utilisateur et la phrase secrète protégeant sa clé privée. Une empreinte reçue par le réseau suffit-elle, seule, à établir la confiance ?

**P2 Preuve à conserver** Empreinte publique du serveur comparée à la console, puis commande id réussie par clé en IPv4 et IPv6. Aucune clé privée ni phrase secrète dans le rapport.

## 3 Politique d'accès du serveur SSH

### 3.1 Configuration et contrôle syntaxique

Sur le serveur, sauvegarder la configuration. Examiner les fichiers inclus et les blocs Match : la première valeur applicable prévaut pour de nombreuses directives ; un nom de fichier tardif ne garantit donc pas la priorité [R2].

```
sudo cp -a /etc/ssh/root/ssh.tp2.before
sudo grep -nE "(Include|Match)" /etc/ssh/sshd_config
sudo ls -l /etc/ssh/sshd_config.d
sudo tee /etc/ssh/sshd_config.d/00-tp2.conf >/dev/null <<'EOF'
PubkeyAuthentication yes
PasswordAuthentication no
KbdInteractiveAuthentication no
AuthenticationMethods publickey
PermitRootLogin no
AllowGroups sshadmin
AllowAgentForwarding no
AllowTcpForwarding no
LogLevel VERBOSE
EOF
sudo /usr/sbin/sshd -t
sudo /usr/sbin/sshd -T -C \
  user=tpadmin,host=client,addr=10.77.0.20,laddr=10.77.0.10,lport=22
```

### 3.2 Validation de la politique et essais

Dans la sortie effective, vérifier chaque valeur du fragment, notamment allowgroups sshadmin. Refaire la dernière commande avec addr=fd42:77::20 et laddr=fd42:77::10. Corriger à la console toute directive antérieure ou tout bloc Match contradictoire, puis relancer sshd -t avant sudo systemctl reload ssh.service.

Depuis le client, refaire les deux accès légitimes de la section 2 dans de nouvelles connexions. Puis réaliser ces deux témoins négatifs ; tous deux doivent être refusés.

```
ssh -o PreferredAuthentications=password \  
-o PubkeyAuthentication=no tpadmin@10.77.0.10  
ssh -o BatchMode=yes -i ~/.ssh/id_ed25519_tp2 \  
-o IdentitiesOnly=yes tpexterne@10.77.0.10
```

Sur le serveur, consulter `sudo journalctl -u ssh.service --since "-10 minutes" --utc --no-pager` et `id tpexterne`. Relier les refus à la méthode interdite et au groupe absent. Le seul échec d'un accès root sans clé installée ne démontrerait pas `PermitRootLogin`.

### 3.3 Analyse et preuves

**Q3** Que prouvent respectivement `sshd -t`, `sshd -T -C` et une nouvelle connexion réelle ? Pourquoi conserver le témoin `tpexterne` avec une clé valide ?

**P3 Preuve à conserver** Valeurs effectives IPv4/IPv6 et tableau de quatre résultats : deux accès légitimes, mot de passe refusé, compte hors groupe refusé ; deux extraits de journal suffisent.

## 4 Filtrage IPv4 et IPv6 avec nftables

### 4.1 Règles IPv4 et IPv6

À la console serveur, vérifier de nouveau que nft list ruleset est vide et UFW inactif ; sinon restaurer l'instantané propre. Relever les vrais noms des cartes interne et NAT. Le fichier ci-dessous gère uniquement la table pédagogique tp2 et autorise des connexions SSH depuis deux adresses exactes [R5, R6].

```
read -r -p "Carte interne du serveur : " LAB_IF
read -r -p "Carte NAT du serveur : " NAT_IF
ip link show dev "$LAB_IF"
ip link show dev "$NAT_IF"
sudo install -d -m 755 /etc/nftables.d
sudo tee /etc/nftables.d/tp2.nft >/dev/null <<EOF
flush table inet tp2
table inet tp2 {
    chain input {
        type filter hook input priority 0; policy drop;
        iifname "lo" accept
        ct state invalid counter drop
        ct state established,related accept
        iifname "$NAT_IF" udp sport 67 udp dport 68 accept
        ip protocol icmp accept
        meta l4proto ipv6-icmp accept
        iifname "$LAB_IF" ip saddr 10.77.0.20 tcp dport 22 counter accept
        iifname "$LAB_IF" ip6 saddr fd42:77::20 tcp dport 22 counter accept
        counter drop
    }
    chain forward {
        type filter hook forward priority 0; policy drop;
    }
    chain output {
        type filter hook output priority 0; policy accept;
    }
}
EOF
```

### 4.2 Trafic de contrôle et résultats attendus

La famille inet traite IPv4 et IPv6. Les réponses aux connexions sortantes et le renouvellement DHCPv4 du NAT restent permis. ICMPv6 transporte notamment la découverte des voisins et les messages de MTU ; le TP en conserve l'ensemble.

**Q4** Prédire les résultats pour .20, .30, ::20 et ::30. Quelle panne pourrait provoquer le blocage général d'ICMPv6, même si TCP 22 était autorisé ?

## 5 Validation des décisions de filtrage

### 5.1 Chargement et contrôle des règles

À la console serveur, préparer la table avant la première lecture, valider puis charger le fichier. Ne jamais exécuter `nft flush ruleset` dans ce TP.

```
sudo nft add table inet tp2
sudo nft -c -f /etc/nftables.d/tp2.nft
sudo nft -f /etc/nftables.d/tp2.nft
sudo nft list chain inet tp2 input
```

### 5.2 Essais depuis les sources autorisées et interdites

Sur le client, ajouter deux sources témoins sur la seule carte interne. Attendre que les adresses IPv6 ne soient plus tentatives ; arrêter si `dadfailed` apparaît. Chaque essai impose explicitement sa source.

```
read -r -p "Carte interne du client : " LAB_IF
sudo ip address add 10.77.0.30/24 dev "$LAB_IF"
sudo ip -6 address add fd42:77::30/64 dev "$LAB_IF"
ip -6 addr show dev "$LAB_IF"
nc -4 -vz -w 3 -s 10.77.0.20 10.77.0.10 22
nc -4 -vz -w 3 -s 10.77.0.30 10.77.0.10 22
nc -6 -vz -w 3 -s fd42:77::20 fd42:77::10 22
nc -6 -vz -w 3 -s fd42:77::30 fd42:77::10 22
ssh -b 10.77.0.20 -i ~/.ssh/id_ed25519_tp2 \
-o IdentitiesOnly=yes tpadmin@10.77.0.10 id
ssh -b fd42:77::20 -i ~/.ssh/id_ed25519_tp2 \
-o IdentitiesOnly=yes tpadmin@fd42:77::10 id
```

Les sources `.20` et `::20` doivent réussir ; `.30` et `::30` doivent expirer avant l'authentification. À la console serveur, comparer les compteurs de la chaîne `input` avant/après, puis vérifier la sortie NAT avec `sudo apt update`. Une session déjà ouverte ne prouve pas qu'une nouvelle connexion est autorisée.

**Q5** Comment distinguer une décision du filtre réseau d'un refus d'authentification SSH ? Donner deux observations complémentaires, sans se limiter au message du client.

**P4 Preuve à conserver** Matrice des quatre sources et résultats TCP, deux identités SSH, compteurs et contrôle de sortie NAT. Reprendre ce même tableau dans le rapport.

### 5.3 Récupération de l'accès

Récupération immédiate : à la console serveur, `sudo nft delete table inet tp2` retire uniquement le filtre du TP. Corriger le fichier, recréer la table et recharger. Les adresses et les règles actives sont temporaires ; la persistance n'est pas demandée dans les manipulations guidées.

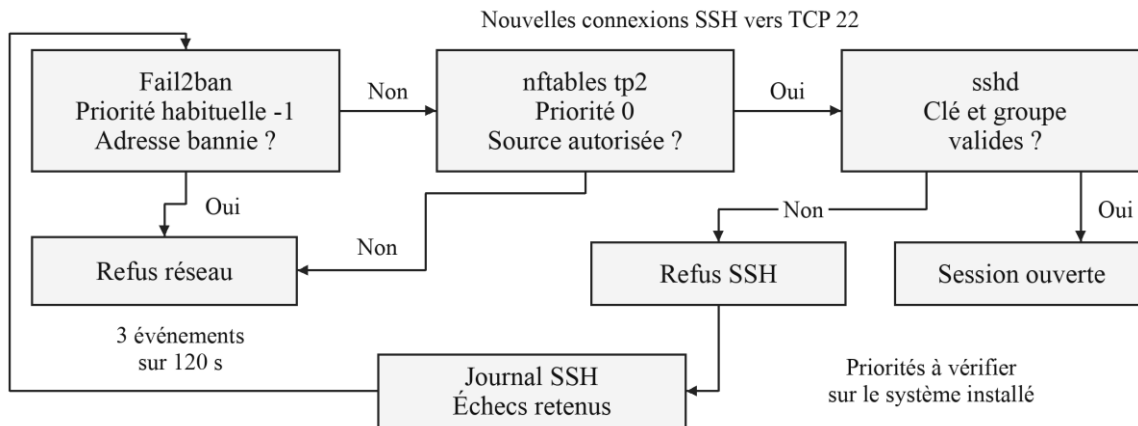
## 6 Détection et réponse automatique avec Fail2ban

### 6.1 Paramètres de détection et de bannissement

Sur le serveur, vérifier que les refus précédents datent de plus de 120 secondes afin de ne pas les confondre avec l'essai suivant. Créer la surcharge locale : backend systemd lit le journal et ne reçoit pas de logpath [R7].

```
sudo tee /etc/fail2ban/jail.d/90-tp2.local >/dev/null <<'EOF'
[sshd]
enabled = true
backend = systemd
filter = sshd[mode=normal]
port = 22
banaction = nftables-multiport
findtime = 120
maxretry = 3
bantime = 180
ignoreip = 127.0.0.1/8 ::1
usedns = no
EOF
sudo fail2ban-client -t
sudo systemctl enable --now fail2ban
sudo fail2ban-client status sshd
sudo fail2ban-client get sshd journalmatch
```

### 6.2 Articulation des contrôles réseau et SSH



**Figure 2 Deux décisions réseau et une décision d'identité**

La table de bannissement peut apparaître au premier ban. Observer ensuite sa priorité réelle dans nft list ruleset : un accept dans une chaîne n'annule pas un drop imposé par une autre [R6, R8].

**Q6** Lire la figure 2 : pourquoi une clé valide ne traverse-t-elle pas un bannissement ? Pourquoi ne faut-il pas placer l'adresse du client dans ignoreip pour cette expérience ?

## 7 Analyse du bannissement et rétablissement de l'accès

### 7.1 Déclenchement et observation du bannissement

À la console serveur, vérifier avec `getent passwd tp2inconnu` que ce compte est absent. Depuis le client, exécuter une fois la commande suivante, puis contrôler le compteur serveur. Répéter au maximum trois fois, avec quelques secondes d'intervalle, et arrêter dès le ban. Tous les essais ciblent exclusivement votre VM.

```
ssh -b 10.77.0.20 -o BatchMode=yes -o ConnectTimeout=3 \
-o IdentitiesOnly=yes -i ~/.ssh/id_ed25519_tp2 \
tp2inconnu@10.77.0.10 true
```

Après chaque essai, à la console serveur, relier l'événement SSH à l'adresse bannie et à la règle réseau. Le nombre de connexions peut différer du nombre d'événements retenus par le filtre.

```
sudo journalctl -u ssh.service --since "-3 minutes" \
--utc --no-pager -o short-iso
sudo fail2ban-client status sshd
sudo nft list ruleset
```

### 7.2 Contrôle du blocage et rétablissement

Pendant le ban, depuis le client, essayer le compte légitime avec sa clé en imposant `.20`. Il doit être bloqué avant l'authentification ; vérifier aussi que la commande IPv6 de la section 5, liée à `::20`, reste possible.

```
ssh -b 10.77.0.20 -o BatchMode=yes -o ConnectTimeout=3 \
-o IdentitiesOnly=yes -i ~/.ssh/id_ed25519_tp2 \
tpadmin@10.77.0.10 id
```

À la console serveur, lever le ban avec les commandes suivantes, puis reprendre la dernière connexion IPv4 : elle doit réussir [R9].

```
sudo fail2ban-client set sshd unbanip 10.77.0.20
sudo fail2ban-client status sshd
```

Si aucun ban n'apparaît après trois essais, ne pas multiplier les tentatives. Examiner `journalmatch`, le journal SSH et `sudo fail2ban-regex systemd-journal "sshd[mode=normal]"`. Expliquer le diagnostic ; ne pas inventer de résultat.

### 7.3 Limites et preuves

**Q7** Quelles limites subsistent si plusieurs utilisateurs partagent une adresse NAT ou si une source autorisée est compromise ? Le seul statut active de Fail2ban prouve-t-il le fonctionnement observé ?

**P5 Preuve à conserver** Une chronologie courte : événement, compteur/adresse bannie, refus du compte légitime, maintien IPv6, puis réussite après déban.

## 8 Tâche autonome d'accès temporaire

Un second poste d'administration doit accéder au même serveur pendant une intervention. Le client existant le simulera avec 10.77.0.21 et fd42:77::21. Adapter le filtrage et prouver ses effets, sans nouvelle procédure de commandes. Conserver le compte tpadmin, la clé et les contrôles SSH déjà vérifiés. La recherche en ligne est autorisée ; justifier toute solution retenue.

### 8.1 Critères d'acceptation

- A1 Ajouter temporairement les deux adresses de maintenance au client et les seules autorisations SSH nécessaires sur le serveur. Démontrer id par SSH depuis .21 et ::21, avec source explicitement imposée ; le filtrage par défaut et ICMPv6 restent opérationnels.
- A2 Démontrer que .30 et ::30 restent refusées et que .20 reste autorisée. Fournir une matrice de cinq essais comprenant les deux réussites A1 ; faire vérifier un résultat par un autre membre du groupe.
- A3 Révoquer les deux autorisations de maintenance et prouver, avec de nouvelles connexions, le refus depuis .21 et ::21 et le maintien depuis .20. Conserver un extrait des seules règles modifiées et expliquer le rôle des états de connexion.

Aucun nouveau compte, nouvelle clé, second bannissement ou redémarrage n'est demandé. Dans le rapport PDF, réutiliser la matrice de tests plutôt que multiplier les captures : configuration appliquée, source, résultat et conclusion doivent être lisibles.

### 8.2 Synthèse et remise en état

Rédiger une conclusion de 5 à 8 lignes : distinguer confiance dans le serveur, autorisation du compte, filtrage statique et réponse aux événements. Indiquer un résultat vérifié et une limite. Conserver les preuves utiles avant de restaurer les instantanés des deux VM ; ne remettre aucune clé privée.

Si une récupération est nécessaire avant restauration, à la console serveur : `sudo systemctl stop fail2ban` puis `sudo nft delete table inet tp2`. Pour SSH, comparer `/etc/ssh` à `/root/ssh.tp2.before`, corriger à la console, valider par `sshd -t` et recharger `ssh.service`. Ne pas écraser aveuglément la configuration. Sur le client, terminer l'agent par `ssh-agent -k` dans son terminal.

### 8.3 Prolongements facultatifs

- Concevoir la persistance du réseau et du filtrage, puis définir les tests à effectuer après un redémarrage [R3, R4]. Cette persistance n'a pas été validée dans le parcours guidé.
- Comparer un MFA vérifié par le serveur et des certificats utilisateurs SSH : confiance, expiration et révocation. Distinguer la phrase secrète locale d'un facteur contrôlé à distance [R2].

## 9 Rapport et évaluation

### 9.1 Consignes de rendu

Remettez un seul rapport PDF par groupe de 2 ou 3 étudiants. Répartissez puis alternez les rôles de réalisation, de vérification et de rédaction. Tous les membres doivent pouvoir expliquer les résultats.

Le rapport comprend les réponses numérotées, les preuves sélectionnées et le résultat de la tâche autonome. Les scripts ou configurations créés peuvent être joints séparément si leur contenu est utile. Aucune archive ni organisation en dossiers n'est demandée.

Ne joignez aucun mot de passe, clé privée, jeton, en-tête LUKS ou donnée personnelle réelle.

### 9.2 Résultats des manipulations guidées

Chaque preuve associe la commande ou la configuration, un résultat lisible et une courte interprétation. Expliquez également les refus attendus. Rédigez au fil des manipulations et retenez uniquement les extraits utiles.

| Preuve | Contenu attendu                                                   |
|--------|-------------------------------------------------------------------|
| P1     | Topologie réelle, adresses et deux tests ping                     |
| P2     | Empreinte d'hôte vérifiée et identités SSH dans les deux familles |
| P3     | Valeurs SSH effectives et deux témoins négatifs expliqués         |
| P4     | Matrice de quatre sources, compteurs et sortie NAT                |
| P5     | Chronologie du ban IPv4, maintien IPv6 et reprise après déban     |

### 9.3 Tâche autonome et synthèse

- A1 Deux nouvelles sources autorisées et identités distantes vérifiées.
- A2 Matrice de cinq essais, comprenant deux sources toujours refusées.
- A3 Révocation dans les deux familles et maintien de l'accès initial.

Concluez par une limite observée et une amélioration justifiée. Signalez tout test non réalisé et son blocage. Une capture de statut ou un score d'audit sans analyse ne suffit pas.

### 9.4 Critères d'évaluation

| Critère                                       | Points |
|-----------------------------------------------|--------|
| Manipulations guidées et preuves exploitables | 7      |
| Réponses argumentées et diagnostic            | 4      |
| Tâche autonome et tests des exigences         | 6      |
| Clarté et rigueur du rapport                  | 3      |
| Total                                         | 20     |

## 10 Références techniques

Ces références complètent les manuels installés dans les VM. Vérifiez les options dans la version locale avant de transposer une configuration. La recherche documentaire est autorisée, y compris pour la tâche autonome. Citez les ressources utilisées et expliquez leur adaptation.

Les figures de cet énoncé sont des schémas pédagogiques originaux.

### R1 Canonical OpenSSH server

<https://ubuntu.com/server/docs/how-to/security/openssh-server/>

### R2 OpenSSH Manuel sshd\_config

[https://man.openbsd.org/sshd\\_config](https://man.openbsd.org/sshd_config)

### R3 Netplan Essai et retour arrière netplan try

<https://netplan.readthedocs.io/en/stable/netplan-try/>

### R4 Netplan Référence du format YAML

<https://netplan.readthedocs.io/en/stable/netplan-yaml/>

### R5 Projet nftables Politique pour un serveur

[https://wiki.nftables.org/wiki-nftables/index.php/Simple\\_ruleset\\_for\\_a\\_server](https://wiki.nftables.org/wiki-nftables/index.php/Simple_ruleset_for_a_server)

### R6 Projet nftables Chaînes et priorités

[https://wiki.nftables.org/wiki-nftables/index.php/Configuring\\_chains](https://wiki.nftables.org/wiki-nftables/index.php/Configuring_chains)

### R7 Ubuntu 24.04 Manuel jail.conf de Fail2ban

<https://manpages.ubuntu.com/manpages/noble/man5/jail.conf.5.html>

### R8 Projet Fail2ban Action nftables

<https://raw.githubusercontent.com/fail2ban/fail2ban/master/config/action.d/nftables.conf>

### R9 Ubuntu 24.04 Manuel fail2ban-client

<https://manpages.ubuntu.com/manpages/noble/man1/fail2ban-client.1.html>